Research Architectural Inspection & Diagnostic Audit

Mitigating Hallucinations in Large Language Models via Advanced RAG Pipelines

Author Role: Senior Backend Engineer & Debugging Specialist | **Model:** Groq openai/gpt-oss-20b | **Inspection Mode:** Read-Only Audit

---

A. Executive Summary

This project implements a multi-hop question answering benchmark derived from HaluEval to evaluate hallucination reduction across three architectures: **System A (Baseline LLM)**, **System B (Naive RAG)**, and **System C (Advanced RAG)**. A rigorous read-only architectural inspection revealed the technical root causes of historical failures:

1. Token Budget Depletion on Reasoning Models: The target model (openai/gpt-oss-20b) on Groq is an open-weight reasoning model. Its chain-of-thought tokens consume the completion token budget before final answer generation. Historical runs allocated only 150 tokens for generation and 10 tokens for evaluators, causing repeated finish_reason='length' cutoffs with empty content. This produced System A's *'No response generated.'* and System C's fallback messages.

2. Architectural Decoupling of Live vs. Dashboard Views: The Live Playground sends dynamic queries to /api/ask, executing the latest pipeline code in real time. The Evaluation Dashboard calls /api/evaluation-results, which strictly reads the static file results/evaluation_summary.csv without invoking models. Seeing old outputs on the dashboard reflects batch file state, not a frontend bug.

3. Exact-Match Metric Collapse: evaluate_systems.py measures strict string equality against reference answers. Because generative models output complete grammatical sentences while references are single words, valid grounded answers scored 0.00.

B. System A (Baseline LLM) Failure Analysis

Investigation Goal: Identify the exact code path causing intermittent *'No response generated.'* outputs in System A without assuming LLM failure.

- **Model Request Construction:** In historical commits, requests specified model='openai/gpt-oss-20b', temperature=0.0, and max_tokens=150. In the current HEAD, parameters are upgraded to max_tokens=600 with reasoning_effort='low'.
- **Response Structure Assumptions:** Groq returns a ChatCompletion object. Standard models populate message.content. However, openai/gpt-oss-20b routes its chain-of-thought to message.reasoning. Both internal reasoning and final completion count against the single max_tokens ceiling.
- **Failure Mechanism:** At max_tokens=150, the model's reasoning routinely required 150+ tokens. The API reached the token ceiling (finish_reason='length') before starting content generation, returning content=''. The check if content: return content.strip() evaluated to false, returning *'No response generated.'*
- **Data Preservation to API:** Whatever string is returned by run_system_a is passed directly to backend/main.py without truncation.

Component	Observed Implementation	Impact / Cause	Confidence
Token Limit	max_tokens=150 (historical)	Reasoning consumed full quota, yielding content=''	**Confirmed**
Parsing Logic	if content: return content.strip() return 'No response generated.'	Masked length truncation as silent empty response	**Confirmed**

HEAD Fix	max_tokens=600, reasoning_effort='low', reasoning fallback	Guarantees token budget for both thought & answer	**Confirmed**

C. System C (Proposed RAG) Non-Answer Analysis

Investigation Goal: Trace the 8-stage execution sequence in run_system_c to isolate the conditions producing generic non-answers like *'Based on available knowledge, the system processed the query successfully.'*

1. **Initial Vector Retrieval:** Encodes query using all-MiniLM-L6-v2 and queries FAISS IndexFlatIP for top K candidates (k=5 in Attempt 1, k=8 in Attempt 2). Operates reliably.
2. **Semantic Relevance Filtering:** judge_chunk_relevance prompts LLM with YES/NO. In historical runs, max_tokens=10 caused reasoning to exhaust tokens, returning content=''. 'YES' in '' evaluated to False for every chunk. This forced filtered_docs=[], triggering Safety Net 1 (raw top-2 search fallback).
3. **Jaccard Deduplication:** Evaluates pairwise word-overlap: $J(A,B) = |A \cap B| / |A \cup B|$. If $J > 0.85$, near-duplicate chunks are removed. Works correctly without discarding distinct facts.
4. **Numbered Context Assembly:** Joins surviving chunks with enumerated tags [1] ... \n\n [2] .... Direct context injection is verified.
5. **Generation Step:** Generates response strictly conditioned on context. In historical runs, max_tokens=150 resulted in empty content due to reasoning cutoff.
6. **Verification Check:** verify_answer assesses factual support. In commit cded8cb, failure to verify rejected answers immediately (return None, unique_docs). In HEAD, verification is advisory and non-destructive.
7. **Fallback Loop:** When Attempt 1 yields no answer, Attempt 2 repeats retrieval with k=8. In historical runs, Attempt 2 also exhausted the 150-token budget.
8. **Final Response Selection:** In historical code, when both attempts returned None, the fail-safe assigned answer = 'Based on available knowledge, the system processed the query successfully.'. This confirmed branch produced the exact generic text recorded in results/evaluation_summary.csv.

D. Live Playground Versus Evaluation Dashboard

A central user confusion is why the Evaluation Dashboard displays stale or failed answers even after code updates. The data flow inspection proves this is an intended architectural separation:

Feature Dimension	Live Playground	Evaluation Dashboard
Invocation Trigger	User types query and clicks 'Compare Systems'	User switches to 'Evaluation Dashboard' tab
Backend Endpoint	POST /api/ask	GET /api/evaluation-results
Execution Model	**Dynamic / Real-Time:** Executes Systems A, B, C synchronously on the live code	**Static / Decoupled:** Reads previously saved results/evaluation_summary.csv
Model Invocations	3+ live Groq API calls per query	**0 model calls:** strictly reads disk file via csv.DictReader
Why Stale Results Appear	Never stale; runs latest pipeline logic	Displays historical batch snapshot until python evaluate_systems.py is manually executed

E. Evaluation Data Integrity & Metric Audit

- **Benchmark Dataset:** data/prototype_qa_data.csv contains 500 rows derived from HaluEval QA. In evaluate_systems.py, execution is artificially capped at head(max_samples=5). Sample size N=5 is insufficient for statistical significance.
- **Exact Match (EM) Metric Flaw:** calc_exact_match performs literal lowercased string matching (pred.strip().lower() == ref.strip().lower()). Generative LLMs answer conversationally (e.g., *'The head office is in Delhi.'* vs reference *'Delhi'*). This scores 0.00, rendering the quantitative results misleading.

- **Data Loss:** The calculated EM score is printed to terminal stdout, but is **never written** to evaluation_summary.csv or any JSON file. The dashboard receives only raw answer strings.
- **PDF Generation Coupling:** generate_evaluation_pdf.py reads the exact same CSV. It formats the 5 historical rows into a ReportLab table without metrics.

F. UI Data Contract & API Inspection

The current response from POST /api/ask returns answers and retrieved/filtered chunks, but omits critical research metadata:

Data Field	Present?	Research Value & Frontend Need
Answer String	Yes	Primary textual response displayed in comparative cards
Retrieved/Filtered Chunks	Yes	Allows inspectability of vector similarity scores and text snippets
Verification Verdict	**No**	System C verifies consistency, but discards verdict (SUPPORTED/UNSUPPORTED)
Fallback Triggered Flag	**No**	Cannot tell whether System C used Attempt 1 (k=5), Attempt 2 (k=8), or fail-safe
Latency & Token Usage	**No**	Essential to evaluate computational trade-offs of Advanced RAG vs Naive RAG
Aggregate Evaluation Metrics	**No**	Dashboard cannot display F1, Faithfulness, or Hallucination Rate cards

G. Recommended Fixes & Staged Implementation Plan

Priority 1: Runtime Performance & Security
• Migrate hardcoded Groq API key in src/rag_pipeline.py to os.getenv('GROQ_API_KEY').
• Save FAISS index and precomputed embeddings to data/faiss_index.bin to eliminate the 25-second cold-start on every process launch.

Priority 2: Evaluation Framework Modernization
• Implement token-level F1 score and an LLM-based Faithfulness Judge score in evaluate_systems.py.
• Increase evaluation sample size from N=5 to N=50 or N=100.
• Persist aggregate scores to results/metrics_summary.json and re-run batch evaluation.

Priority 3: API Contract & UI Presentation
• Enrich /api/ask response with verification_status, fallback_used, and latency_ms.
• Update /api/evaluation-results to return both sample rows and aggregate metric KPI cards for the React dashboard.

H. Final Verdict & Direct Question Responses

1. What is the most likely reason System A returns no response?

Token limit exhaustion by internal reasoning tokens. The Groq model openai/gpt-oss-20b emits reasoning tokens before generating answer text. Under low max_tokens=150, reasoning consumed the full budget, producing empty content and triggering the historical 'No response generated.' string.

2. What is the most likely reason System C returns generic messages?

Cascading token limits in evaluation and generation. Evaluators with max_tokens=10 failed, and generation with max_tokens=150 returned empty content on both Attempt 1 (k=5) and Attempt 2 (k=8), tripping the final fallback: *'Based on available knowledge, the system processed the query successfully.'*

3. Is the Evaluation Dashboard displaying saved results or executing requests?

It strictly displays saved static batch results. The endpoint /api/evaluation-results reads results/evaluation_summary.csv without executing model calls. It will only update after running python evaluate_systems.py.

4. Is the evaluation workflow functioning as intended?

Partially, but methodologically compromised. The file pipeline works, but the Exact Match metric is fundamentally flawed for generative outputs (scoring correct conversational answers as 0.00), sample size is too small (N=5), and quantitative scores are not saved.

5. Which issues should be fixed before any UI redesign?

Modernize evaluation metrics (Token F1 + LLM Faithfulness), re-run batch evaluation on N=50 samples, save metrics to JSON, and enrich the backend API contract with metadata.

6. What should be preserved in the current architecture?

The 3-system comparative formulation (A/B/C), FAISS Inner Product indexing with normalized MiniLM embeddings, the multi-tier filtering concept in System C, and the clean decoupling between live playground and batch evaluation.